

Intelligent Analysis of Technical Support Requests in Service Desk Ticketing Systems

A. A. Gorbunova, A. S. Trunov, V. I. Voronov

Moscow Technical University of Communications and Informatics

Moscow, Russia

19alinalinalinalina97@gmail.com; greek17@yandex.ru; vorvi@mail.ru

Abstract— Natural language processing methods are actively being used in systems that interact with people. These systems include voice assistants, spam filters, Service Desk ticket processing systems and many others. This article studies the problem of intelligent automatization of request processing. Requests are coming from users and describe a problem in wireless network equipment. The subsystem described in the article is integrated via specifically developed API to the Atlassian Jira technical support system to access user requests. Requests in technical support system contain information about a problem in the wireless network that the user encountered. The main goal of the subsystem is to analyze the input message, which is written in natural language. Methods of intelligent natural language text processing extract information about a network problem that can help locate a specific device and determine the cause of the problem, which can then be fixed by standard actions. A word embedding approach called word2vec is used for text analysis. The article describes the work process of convolutional neural networks and their integration with natural language processing methods for obtaining needed results, key modules that make up the subsystem, in particular, the message processing subsystem and the actualizer system and the relationship between them. Examples of input message text analysis and results obtained by the subsystem are shown.

Keywords— Natural language processing; convolutional neural network; word embedding; word2vec; Jira API; classification

I. INTRODUCTION

With the growing popularity of voice-user interfaces and chatbots, the natural language processing (NLP) has become one of the key data mining technologies. It is widely used for search engine optimization, ad targeting, translation systems, sentiment analysis, and many other areas. Some examples of newly developed technologies that use natural language processing are:

- Cortana [1]. A virtual assistant in Windows that can process speech. With Cortana, you can create reminders, open apps, send emails, play games, check the weather, etc.
- Siri [2]. An assistant for Apple OS (iOS, watchOS, macOS, HomePod, and tvOS). Some functions that use voice control are: call / write someone, send a letter, set a timer, take a photo, etc.
- Gmail [3]. A well-known mail service can detect spam so that it does not end the inbox.

- Dialogflow [4]. Google platform that lets you create NLP bots. For example, a bot for ordering pizza.

Along with large companies, research groups at universities are also involved in solving problems related to the development of intelligent systems for recognizing and processing natural language. Department of ISM&A at MTUCI conducts research on formalizing unstructured text [5], legacy applications integration in Scientific Software System [6, 7, 23, 24], Gesture Language Recognition [8], [9], SMS analysis to prevent riots at mass events [10]. All these technologies are associated with the use of intelligent methods of information processing in communication in various areas of human activity.

II. NATURAL LANGUAGE PROCESSING WITH WORD EMBEDDING

There are many approaches and methods for analyzing natural text that will describe problems in corporate networks. Word embedding is a mapping of an entity to a certain vector.

The easiest way to get a vector from a word is to take a vector of the length of the dictionary and put one unit in the position corresponding to the word number in the dictionary. This approach is called one-hot encoding (OHE). OHE still lacks semantic proximity properties, so the analysis does not use the word's meaning [11].

If we take the OHE vector of each word in the text and add it together (i.e. the output is a simple count of different words in the text in one vector) (Fig. 1). This approach is called a bag of words (BoW), because all information about the mutual relation of words within the text is lost.

	it	is	puppy	cat	pen	a	this
it is a puppy	1	1	1	0	0	1	0
it is a kitten	1	1	0	0	0	1	0
it is a cat	1	1	0	1	0	1	0

Fig. 1. Bag of words example [12]

Further, you can represent the corpus (set of texts) in the form of a "word-document" (term-document) matrix. This matrix leads to dedicated models, where the word-document matrix can be represented as the product of two word-topic

and topic-document matrices. In the simplest case, using SVD decomposition, a matrix is turned into a representation of words through topics and documents through topics. The results are very dependent on the given corpus of words [11].

Described approaches are suitable for cases where the number of texts is small and the vocabulary is limited. But with the spread of the Internet and the rapid growth of data for analysis, a lot of texts surpass size limitations, including those with a changing and expanding vocabulary.

In 2013, Czech graduate student Tomáš Mikolov proposed his own approach to word embedding, which he called word2vec [13]. His approach is based on another important hypothesis, which is commonly called the hypothesis of locality - "words that occur in the same environment have similar meanings." Proximity in this case is understood very broadly: only matching words can stand nearby. Based on this hypothesis, Tomasz Mikolov proposed a new approach that did not suffer from large amounts of information, but rather used them.

The model proposed by Mikolov is very simple - predicting the probability of a word from its environment (context). Vectors of words are composed in such a way so that the probability of a word assigned by the model is close to the probability of meeting this word in this environment in a real text.

$$P(w_0 | w_c) = \frac{e^{s(w_0, w_c)}}{\sum_{w_i \in V} e^{s(w_i, w_c)}} \quad (1)$$

Where w_0 — target word vector, w_c — a context vector, calculated (for example, by averaging) from vectors surrounding the desired word of other words and $s(w_1, w_2)$ — a function that maps two vectors into one number.

The equation (1) is called softmax, which is "soft maximum", soft - in the sense of differentiable. It is necessary so that the model can learn using backpropagation, which is the process of backpropagation of an error.

The training process is structured as follows: $(2k + 1)$ words are taken sequentially, the word in the center is the word that should be predicted and the surrounding words are the context of length k on each side). Each word in the model is associated with a unique vector, which we change in the process of training the model.

In general, this approach is called CBOW - continuous bag of words, continuous because sets of words from the text are sequentially fed into the model, and BoW because the word order in the context is not important.

Mikolov also proposed a different approach - the exact opposite of CBOW, which he called skip-gram, which is "a phrase with a skip." The context of this word is guessed. The rest of the model remains unchanged.

It is worth noting that although the model does not explicitly contain any semantics, only the statistical properties

of text corpora, it turns out that a trained word2vec model can capture some semantic properties of words. An example of work is shown in Fig. 2.

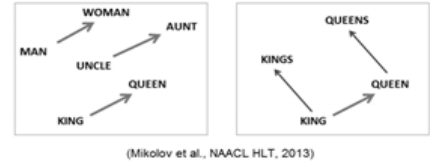


Fig. 2. Example of CBOW [11]

The described CBOW standard model predicts the probability of words and their optimization. The function for optimization is called the Kullback-Leibler divergence (2).

$$KL(p || q) = \sum_{x \in V} p(x) \log \frac{p(x)}{q(x)} \quad (2)$$

It turned out that it is quite difficult to optimize this formula. Primarily due to the fact that $q(x)$ is calculated using softmax throughout the dictionary. It is worth noting here that many words do not occur together, so most of the computation in softmax is redundant. An alternative approach was proposed, which was called Negative Sampling. The essence of this approach is that it maximizes the likelihood of having the desired word in a typical context (the one that is often found in the corpus) and at the same time minimizes the likelihood of having it in an atypical context (the one that occurs rarely or not at all) (3).

$$NegS(w_0) = \sum_{i=1, x_i \in D}^{i=k} -\log(1 + e^{-s(w_i, w_0)}) + \sum_{j=1, x_j \notin D}^{j=l} -\log(1 + e^{-s(w_j, w_0)}) \quad (3)$$

The positive part is responsible for typical contexts, and D is the distribution of the joint occurrence of the word w and the rest of the words in the corpus. The negative part is a set of words that rarely occur with the target word. When optimized, this function leads to a result similar to the standard softmax.

It is not necessary to change the original formula, the softmax itself can be calculated more efficiently. For example, using a binary tree. A Huffman tree can be build for all words in the dictionary. In the resulting tree, V words are located on the leaves of the tree (Fig. 3).

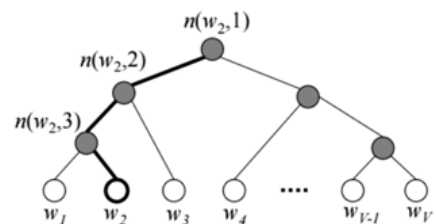


Fig. 3. Example of Huffman Tree [11]

The figure shows an example of such a binary tree. The path from the root to the word w_2 is highlighted in bold. The length of the path is $L(w)$, and the j -th vertex on the path to the word w is denoted by $n(w, j)$.

Using the hierarchical softmax, vector $v_{n(w,j)}$ is predicted for $V-1$ interior vertices. The equation (4) calculates the probability of the word w being the output word (depending on what we predict: a word from the context or context from the word).

$$p(w = w_0) = \prod_{j=1}^{L(w)-1} \sigma([n(w, j+1) = lch(n(w, j))]v_{n(w,j)}^T u) \quad (4)$$

When using a simple softmax to calculate the probability of a word, it was necessary to calculate the normalization sum for all words from the dictionary and it took $O(V)$ operations. Now the word probability can be computed using sequential computations that require $O(\log(V))$.

In the text above, it was implied that different forms of the same word are considered different words and that the model will be good enough to learn their syntactic proximity, or stemming or lemmatization mechanisms would be used. Stemming is cutting off the end of a word, leaving only the stem. And lemmatization is the replacement of a word with its initial form. But this information can be used instead of getting lost - by encoding the OHE into a new vector, and concatenating it with the vector for the stem or lemma.

III. APPLICATION OF CONVENTIONAL NEURAL NETWORKS FOR NATURAL LANGUAGE PROCESSING TASKS

When we hear about Convolutional Neural Networks (CNNs), we usually think about computer vision. But, as it turns out, CNNs are also good for Natural Language Processing (NLP) tasks. Moreover, a recent article from a team of authors at Intel and Carnegie-Mellon University [14] argue that they are even better suited for NLP than the RNNs, which have reigned the field in recent years.

The idea of a convolutional network is to use convolutional layers, which consist of feature maps and each map has a kernel (filter). The number of cards, like in any other network architecture, is chosen depending on the problem that it solves. It is recommended to have two convolutional layer maps for each map of the previous layer [15].

The kernel is a filter or a window that slides over the entire area of the previous map and finds certain features of objects.

Initially, the values of each map of the convolutional layer are equal to 0. The weight values of the kernels are set randomly in the range from -0.5 to 0.5. The kernel slides over

the previous map and performs the convolution operation, which is often used for image processing (5).

$$(f * g)[m, n] = \sum_{k,l} f[m-k, n-l] * g[k, l] \quad (5)$$

where f – original image matrix and g – convolution kernel.

This operation can be described as follows - we go through the entire image with a given step (usually 1) with a kernel size window, at each step we multiply the contents of the window by the kernel, the result is summed up and written into the result matrix.

For convolutional neural network to work, a matrix of input values is fed to its input. Embedding is the mapping of a point in some multidimensional space to an object, in our case a word. An example of embedding is Word2Vec. So, we take embedding for each word in our text and just put all the vectors in a row, getting the required matrix.

Convolutional neural networks are a good choice where you need to see a piece or the whole sequence and draw a conclusion from this. These are tasks such as spam detection, sentiment analysis, or extraction of named entities [16].

In this work [17], the authors classify texts based on the symbols. They used a 70 character set to represent each character as a one-hot vector and set a fixed text length of 1014 characters. Thus, the text is represented by a 70x1014 binary matrix. The network has no idea about words and sees them as combinations of characters. Information about the semantic proximity of words is not provided to the network, as in the case of pre-trained Word2Vec vectors. The network consists of 1d convoluted max-pooling layers and two fully-connected layers with a dropout. On large datasets, they performed even better than word networks. In addition, this approach greatly simplifies the preprocessing step, which can facilitate its use on mobile devices.

IV. INPUT DATA

The main goal of the system is to analyze error messages that come from users and make a decision on how to correct them. For efficiency, it is necessary to unify the process of sending messages. JIRA Software was chosen as the system which collects messages.

Originally, Jira is a bug tracking system, which is a system for tracking code errors or network activity. Although many companies also use it as a project management tool. It provides the ability to add your own statuses, log time stamps, link tasks between projects, collect statistics and reports. JIRA Software has been chosen as the system through which messages are collected [18].

The JIRA REST API allows you to interact with a third party JIRA applications. The entire list of APIs can be found in the official JIRA documentation [19]. The preferred authentication methods for the JIRA REST API are OAuth

and HTTP Basic (when using SSL). Other supported methods include: HTTP cookies and trusted applications.

In general, an API command call is an HTTP request to an API URL with a method (GET, PUT, POST and DELETE), command, and request body. The body of the request, as well as the API response, are in JSON format. An example of a request that will return a JSON representation of a ticket: GET /rest/api/latest/issue/{ticket_number}. Using the API requests, you can create tickets, edit them, write comments, upload tickets and more [20]. The service interface is shown in Fig. 4.

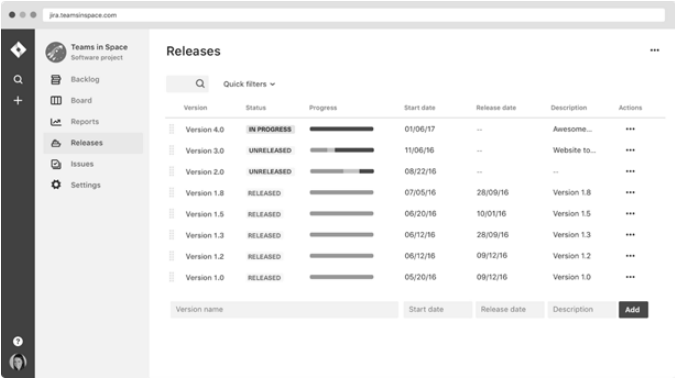


Fig. 4. JIRA Software interface [21]

Within the process, user composes a free-form text describing the error. Some general requirements for a message are:

- Describes one problem;
- Doesn't contain unrelated information;
- Must be described in as much detail as possible;
- To write the most informative message, a list on leading questions is shown to a user.

Processing system receives the message from JIRA Software, uses it as input and, after using data mining methods, gives the device id and error code from a pre-compiled list.

During the training process, a text in the form of a matrix is fed to the input of the neural network. A pair of output values corresponds to each text in the set. For example, after processing the basic text "The router in the office 203 has stopped working" the result of solving the classification problem could be in " 4; 29 ", where 4 is the device code for a router and 29 means "Device not working".

V. FAILURE REQUESTS PROCESSING SYSTEM ARCHITECTURE

The text processing and equipment configuration subsystems are developed within the hardware and software system of network problems request processing. The system is designed to handle natural language requests that were created in Jira Software and determine the root of the network problem. The architecture of the complex is shown in Fig. 5.

The system consists of the following components:

- Subsystem for collecting messages about problems. The Jira Software system, which was described in detail in IV, is used for this. Messages from this system are then passed to the word processing subsystem through the Jira API using the GET method;
- Text processing subsystem provides processing of messages in natural language using the word embedding methods and solves the classification problem. The output is the information about the described problem and the device on which it occurred. The results are sent to the "Interpreter" subsystem;
- Interpreter subsystem is based on the text analysis result and determines what actions can be taken to configure the network, and then sends the result to the equipment configuration subsystem;
- Hardware configuration subsystem receives the interpreter's solution, and sends the necessary configuration commands to the network equipment. It also logs the action on a file server;
- File storage - stores network configuration logs.

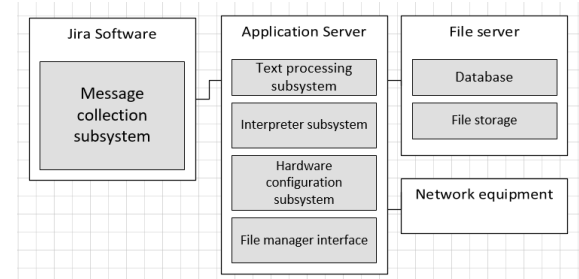


Fig. 5. Architecture of the network fault request processing system

For a visual representation of subsystems' interaction, a generalized class diagram of each subsystem is shown in Fig. 6.

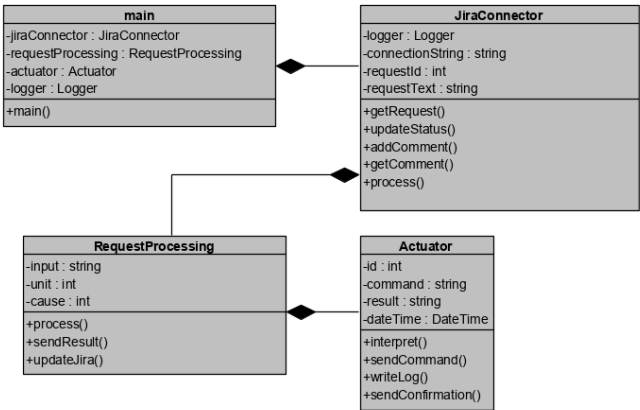


Fig. 6. Generalized class diagram of subsystems.

For a more accurate and detailed description for each subsystem, a separate class diagram should be drawn, which would take into account all the nuances and details of the system.

VI. RESULTS OF THE TEXT ANALYSIS MODELS

To get various results, different models of natural language text analysis will be used. The two different texts would be used as an input: a large and a small one. As a large corpus of text, we will use an article [22], which describes the process of equipment configuration in wireless corporate networks. For a small corpus, the following short text will be used: "the internet stopped working I tried to do router reload and it didn't work, speed is down."

The result of applying the word2vec using a large corpus is shown in Fig. 7.

```
sleepingsquidgy@ubuntu:~$ python word2vec_test.py
[('trigger', 0.29450616240501404), ('reverting', 0.29215508699417114), ('organiz', 0.28976505994796753), ('exploring', 0.26969224214553833), ('allows', 0.26737916469573975), ('education', 0.2610781192779541), ('firmwareaur', 0.25693759322166443), ('wireless', 0.24572336673736572), ('ac', 0.24509507417678833), ('a', 0.2393997311592102)]
```

Fig. 7. Word2vec for a large corpus

The result of applying the word2vec using a small corpus is shown in Fig. 8.

```
sleepingsquidgy@ubuntu:~$ python word2vec_test.py
[(' ', 0.22740605473518372), ('tried', 0.1149425208568573), ('reload', 0.07199451327323914), (' ', 0.06620937585830688), ('speed', 0.06055395305156708), ('n't', 0.057868000119924545), ('internet', -0.02379734441637993), ('working', -0.04645928740501404), ('work', -0.06447864323854446), ('stopped', -0.12796248495578766)]
```

Fig. 8. Word2vec for a small corpus

The figures show which words turned out to be more similar to the word router than the rest. The number shows how close they are based on the input corpus. It can be seen that the model works better for a large corpus of words, where the search word occurs many times.

Results can be obtained separately for the CBOW and Skip-Gram methods. The result of applying the CBOW using a large corpus is shown in Fig. 9

```
Context: ['settings', 'internet', 'reload', 'wireless']
Prediction: networks
```

Fig. 9. Result of CBOW for a large corpus

The results of the CBOW using a small corpus is shown in Fig. 10-12.

```
Context: ['stopped', 'internet', 'reload', 'do']
Prediction: working
```

Fig. 10. Result of CBOW for a small corpus

```
Context: ['stopped', 'internet', 'reload', 'do']
Prediction: reload
```

Fig. 11. Result of CBOW for a small corpus

```
Context: ['stopped', 'internet', 'reload', 'do']
Prediction: router
```

Fig. 12. Result of CBOW for a small corpus

Fig. 10-12 show the word that the model thinks is the most fitting in a given context. All three pictures show that the predicted results may vary.

The result of applying the Skip-Gram using a large corpus is shown in Fig. 13-14.

```
['see', 'rtl', 'device']
```

Fig. 13. Result of Skip-Gram for a large corpus after 1000 epochs

```
['households', 'problems', 'runs']
```

Fig. 14. Result of Skip-Gram for a large corpus after 500 epochs

The result of applying the Skip-Gram using a small corpus is shown in Fig. 15-16.

```
['working', 'work', 'internet', 'speed']
```

Fig. 15. Skip-Gram result for small corpus

```
['tried', 'speed', 'reload', 'working']
```

Fig. 16. Skip-Gram result for small corpus

Figures 15-16 show the words that are most likely to appear in the same context as the word router.

Thus, different models can be used to analyze the text of user requests. The right combination, configuration and optimization of them can give the desired result.

VII. CONCLUSION

The article describes a natural language processing model that will be applied in the intelligent analysis system. This system would process requests in a task-tracking system that describe a problem with the network. Word Embedding and the process of convolutional neural networks are described in relation to text processing. The architecture of the subsystem is developed and its components are described. The article shows some results of applying different models of natural language processing to texts of different sizes.

References

- [1] Microsoft. <https://www.microsoft.com/en-us/cortana>
- [2] Siri, Apple. <https://www.apple.com/siri>
- [3] Gmail, Google. URL: https://www.google.com/intl/en_uk/gmail/about
- [4] Dialogflow, Google. <https://dialogflow.cloud.google.com/#/getStarted>
- [5] Triapitsyn A. D., Larin A. I. «Designing a subsystem for classifying the subject of a news article in a model for formalizing unstructured text information based on word embedding», 2020 International Conference «Engineering Management of Communication and Technology» (EMCTECH), Vienna, Austria, 20-22 October 2020 (unpublished)
- [6] A.S. Trunov, L. I. Voronova, V. I. Voronov, D. I. Sukhachev and V. G. Strelnikov, "Legacy applications model integration to support scientific experiment," 2018 Systems of Signals Generating and Processing in the Field of on Board Communications, Moscow, 2018, pp. 1-7, doi: 10.1109/SOSG.2018.8350641
- [7] A.S. Trunov, L. I. Voronova, V. I. Voronov and D. P. Ayrapetov, "Container Cluster Model Development for Legacy Applications

- Integration in Scientific Software System," 2018 IEEE International Conference "Quality Management, Transport and Information Security, Information Technologies" (IT&QM&IS), St. Petersburg, 2018, pp. 815-819, doi: 10.1109/ITMQIS.2018.8525120
- [8] Artemov, M., Voronov, V., Voronova, L., Goncharenko, A., & Usachev, V. (2019). Subsystem for Simple Dynamic Gesture Recognition Using 3DCNNLSTM. In Conference of Open Innovations Association, FRUCT (No. 24, pp. 571-577). FRUCT Oy
- [9] V. I. Voronov, K. V. Genchel, L. I. Voronova and M. D. Travina, "Development of a Software Package Designed to Support Distance Education for Disabled People," 2018 IEEE International Conference "Quality Management, Transport and Information Security, Information Technologies" (IT&QM&IS), St. Petersburg, 2018, pp. 746-751, doi: 10.1109/ITMQIS.2018.8524958.
- [10] L. I. Voronova, R. V. Tolmachev and A. V. A. Usachev, "Resource development to prevent riots at mass events," 2018 Systems of Signals Generating and Processing in the Field of on Board Communications, Moscow, 2018, pp. 1-5, doi: 10.1109/SOSG.2018.8350580
- [11] Wondrous world of Word Embeddings: what are they and why do we need them? Habr. <https://habr.com/ru/company/ods/blog/329410>
- [12] Chapter 4. The Effects of Feature Scaling: From Bag-of-Words to Tf-Idf, O'Reilly. <https://www.oreilly.com/library/view/feature-engineering-for/9781491953235/ch04.html>
- [13] T. Mikolov, I. Sutskever, K. Chen and D. P. Ayrapetov, "Distributed Representations of Words and Phrases and their Compositionality", Google Scholar, 2013
- [14] S. Bai, J. Z. Kolter, V. Koltun «An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling», Cornell University, 2018
- [15] Using CNN for NLP tasks, Habr. <https://habr.com/ru/company/ods/blog/353060>
- [16] CNN, Part 1: structure, topology, activation functions and training data, Habr. <https://habr.com/ru/post/348000>
- [17] X. Zhang, J. Zhao, Y. LeCun «Character-level Convolutional Networks for Text Classification», Cornell University, 2016
- [18] 10 instruments for project management: efficiency, task distribution and communication, Skillbox. https://skillbox.ru/media/management/10_instrumentov_dlya_pm
- [19] Jira REST APIs, Atlassian. <https://developer.atlassian.com/server/jira/platform/rest-apis>
- [20] Jira configuration for your project. Perfect flow and flawless ticket, Habr. <https://habr.com/ru/company/badoo/blog/424655>
- [21] Jira Software, Atlassian. <https://www.atlassian.com/software/jira>
- [22] Network configuration/Wireless, Archlinux. https://wiki.archlinux.org/index.php/Network_configuration/Wireless
- [23] V.I. Voronov, A.D. Bykov, L.I. Voronova, "Designing a subsystem of face detection and database interface in the hardware-software complex of biometric identification based on neural network face recognition," T-Comm, vol. 14, no.4, pp. 31-38, 2020.
- [24] V.I. Voronov, I.A. Zharov, A.D. Bykov, A.S. Trunov, L.I. Voronova, "Designing a neural network identification subsystem in the hardware-software complex of face recognition," T-Comm, vol. 14, no.5, pp. 69-76, 2020.